



Lab 3: Consistency

1. Overview

In distributed database systems, anything might go wrong. Disks, servers, racks, or data centers may fail. These may result in data loss in a way or another. Therefore, we use replication to extensively decrease the probability of totally losing data. This solution leads to the increased latency to write data to all replicas. As a result, database management systems came up with several solutions to this problem in the form of customizable consistency configurations to suit all needs. For example, we can accept only half the nodes that may be required to write the data to form a quorum. Also, in some database systems, we can allow some read requests to specify the consistency level per request not per schema. This makes it much more flexible to changes in latency requirements.

Apache Cassandra is one of the outstanding tools in this area. Cassandra has a wide range of consistency levels. That's why in this lab we will experiment with tunable consistency levels and replication factors.

2. Installation

For the purpose of this lab, we will use a docker-compose application containing a cluster of **3** cassandra nodes. To set up this cluster, you only need to:

- [Install Docker](#)
- Run `docker-compose -f docker-compose.yml up` on the following [docker-compose.yml file](#)

To execute the **cqlsh (cassandra QL shell)** commands, you should execute this command

```
docker exec -it <container-id> cqlsh
```

To pause a docker container to simulate a node down scenario, you can use the following command:

```
docker container pause <container-id>
```



You can get the container id using:

```
docker ps | grep <container-name>
```

You can also set consistency level per query using the following CQL command:

```
SELECT * FROM bookstore.books USING CONSISTENCY QUORUM WHERE  
name='in to the woods';
```

Notice the `USING CONSISTENCY QUORUM` clause. This can also be used with inserts, updates, and deletes.

3. Requirements

You are required to experiment with replication factors of 1, 2, and 3. For each replication factor, you will test how you will be able to read rows you inserted in case 1 or 2 nodes are down. You will be creating a bookstore KEYSPACE to experiment with. In each experiment, you are required to create the keyspace again with different replication factors. Cassandra partitions data over storage nodes using a special form of hashing called consistent hashing. Cassandra maps every node to one or more tokens (virtual nodes) on a continuous hash ring. (For simplicity purposes we will use only one virtual node for each node and use the Replication strategy of Simple Strategy for data replication within one datacenter).

A) Replication Factor = 1

Below are the instructions you should use to create a table and insert a row using a replication factor = 1.

```
> CREATE KEYSPACE bookstore WITH replication = {'class':  
'SimpleStrategy', 'replication_factor' : '1'};  
  
> CREATE TABLE books (  
name VARCHAR,  
category VARCHAR,  
year INT,  
title VARCHAR,  
PRIMARY KEY ((name), category, year, title)
```



```
) WITH CLUSTERING ORDER BY (category ASC, year DESC, title  
ASC);  
  
> INSERT INTO books (name, category, year, title) VALUES ('in  
to the woods', 'Fiction', 2020, 'In To The Woods');
```

You are required to run the commands above in the following cases:

1. Node 1 down
2. Node 2 down
3. Node 3 down

For each case above, mention if there is any way you can modify consistency to get the query to succeed if it did fail.

B) Replication Factor = 2

Using the same row inserted when RF = 1, we set the replication factor to 2 by modifying replication factor, consistency level, and issuing the query again.

```
> ALTER KEYSPACE "bookstore" WITH REPLICATION =  
{ 'class' : 'SimpleStrategy', 'replication_factor' : 2 };  
  
> CONSISTENCY QUORUM;  
  
> SELECT * FROM bookstore.books WHERE name='in to the woods';
```

You are required to run the commands above in the following cases:

- A) Node 1 down
- B) Node 2 down
- C) Node 3 down
- D) Nodes 1 & 2 down
- E) Nodes 1 & 3 down
- F) Nodes 2 & 3 down

For each case above, mention if there is any way you can modify consistency to get the query to succeed if it did fail.



C) Replication Factor = 3

Using the same row inserted when RF = 1, we set the replication factor to 2 by modifying replication factor, consistency level, and issuing the query again.

```
> ALTER KEYSPACE "bookstore" WITH REPLICATION =  
  { 'class' : 'SimpleStrategy', 'replication_factor' : 3 };  
  
> CONSISTENCY QUORUM;  
  
> SELECT * FROM bookstore.books WHERE name='in to the woods';
```

You are required to run the commands above in the following cases:

- A) Node 1 down
- B) Node 2 down
- C) Node 3 down
- D) Nodes 1 & 2 down
- E) Nodes 1 & 3 down
- F) Nodes 2 & 3 down

For each case above, run and show if there is any way you can modify consistency to get the query to succeed if it did fail.



D) Fill The Matrix Below

After running the experiments, you should decide whether the query was successful **as is** **without any changes** or not. Fill the following table with your results:

Nodes Down	RF = 1	RF = 2	RF = 3
Node 1 down			
Node 2 down			
Node 3 down			
Nodes 1 & 2			
Nodes 1 & 3			
Nodes 2 & 3			

You should also state any needed per request consistency changes to make the query pass **for each case**.

E) Fill The Matrix Below

Repeat the same setup (RF = 1, 2, 3) in requirements A, B, & C, and observe the success/failure for the INSERT statements mentioned above. Then, fill the table below with either success or failure.

Nodes Down	RF = 1	RF = 2	RF = 3
Node 1 down			
Node 2 down			
Node 3 down			
Nodes 1 & 2			
Nodes 1 & 3			
Nodes 2 & 3			

You should also state any needed per request consistency changes to make the query pass **for each case**.



5. Deliverables

You are required to deliver the following:

- A running version of the cassandra cluster on docker
- Screenshots showing terminal output in each of the cases
- Tables in requirement D & E filled in **all** cells with the appropriate answer
- Report containing all of the above.

6. Notes

- INSERT statements should be altered by changing any value for any column to make sure you don't get a select result for old inserts.
- You should work in groups of **three**
- All team members should be ready to answer any question during discussion
- Any cheating will be severely penalized.